

ĐỀ KIỂM TRA 1
MÔN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
ĐỀ SỐ 1

Thời gian làm bài: phút

YÊU CẦU SINH VIÊN ĐỌC KỸ TRƯỚC KHI LÀM BÀI:

1. Tạo một thư mục đặt tên là **MSSV_Họ tên** (viết liền, không dấu).
2. **Sinh viên nhận 03 thư mục Cau1, Cau2 và Cau3**, sao chép 03 thư mục nhận được vào thư mục MSSV_Họ tên vừa tạo.
3. Sau khi làm bài xong, sinh viên nén thư mục MSSV_Họ tên và nộp. Bài nộp đúng là bài nộp có thư mục MSSV_Họ tên chứa 03 thư mục Cau1, Cau2 và Cau3 chứa bài làm.

PHẦN CÂU HỎI

Sinh viên lưu ý đảm bảo bài làm của từng câu phải biên dịch, chạy được và đặt đúng thư mục theo hướng dẫn. Tất cả các trường hợp **làm sai yêu cầu**, sinh **lỗi** trong **quá trình biên dịch** hoặc trong **quá trình chạy** khi chấm đều bị **0 điểm cả câu**.

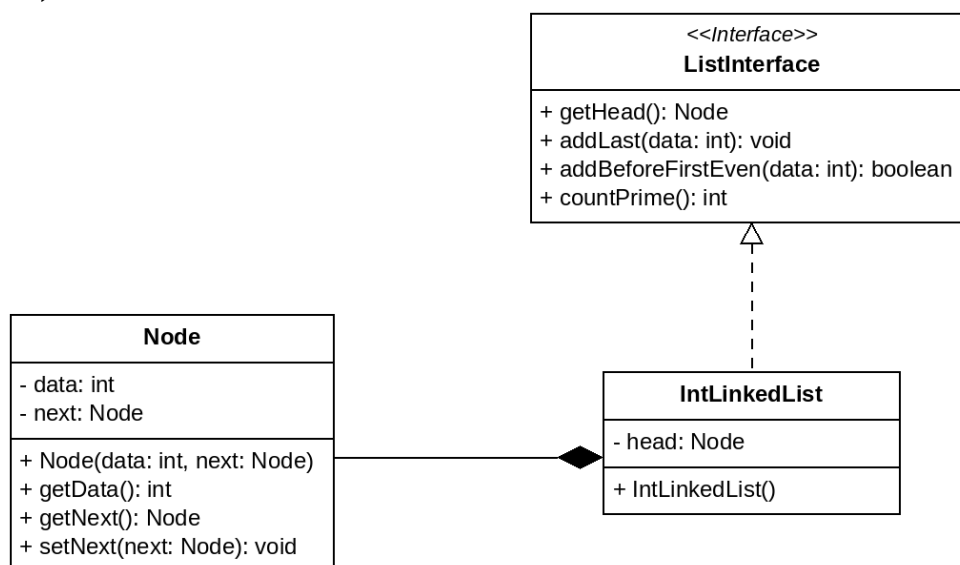
Phần hàm main cho sẵn ở các câu trong đề chỉ để sinh viên kiểm thử bài làm, sinh viên không nhất thiết phải xóa hàm main khi nộp nhưng phải đảm bảo hàm main không gây lỗi cho bài làm.

Sinh viên chú ý **KHÔNG** chỉnh sửa tên file, tên hàm, thứ tự tham số trong các file đã nhận và làm bài trực tiếp trên các file này.

Thư mục bài làm đúng khi hoàn chỉnh có cấu trúc như sau (52100000_NguyenVanAn thay thế bởi thông tin tương ứng của sinh viên):

```
52100000_NguyenVanAn
├── Cau1
│   ├── IntLinkedList.java
│   ├── ListInterface.java
│   ├── Node.java
│   └── Test.java
├── Cau2
│   └── Cau2.java
└── Cau3
    └── Cau3.java
```

Câu 1 (4 điểm):



Trong thư mục Cau1, dựa vào sơ đồ lớp trên và hai file **ListInterface.java** và **Node.java** đính kèm trong thư mục (sinh viên không chỉnh sửa hoặc thêm code trên hai file này), sinh viên định nghĩa lớp **IntLinkedList** tạo ra danh sách liên kết chứa các số nguyên.

Định nghĩa các phương thức trong lớp **IntLinkedList**:

- **IntLinkedList()**: Phương thức khởi tạo không tham số, gán **head = null**.
- **getHead()**: trả về **head** của danh sách liên kết.
- **addLast(int data)**: thêm vào cuối danh sách một nút có giá trị **data**.
Ví dụ: Cho danh sách rỗng, lần lượt gọi addLast(3), addLast(7), addLast(4), addLast(9) thì kết quả danh sách là 3, 7, 4, 9 với **head** là phần tử có giá trị 3.
- **addBeforeFirstEven(int data)**: thêm một nút với giá trị **data** vào **trước** nút chứa số chẵn đầu tiên tính từ đầu danh sách, sau đó trả về **true**. Nếu danh sách không có số chẵn nào thì không thêm và trả về **false**.
Ví dụ: Cho danh sách 3, 7, 4, 9 với **head** là phần tử có giá trị 3. Ta gọi phương thức **addBeforeFirstEven(10)** thì kết quả danh sách sẽ là 3, 7, 10, 4, 9 và phương thức trả về **true**.
- **countPrime()**: trả về số lượng phần tử trong danh sách là số nguyên tố. Số nguyên tố là số nguyên lớn hơn 1, chỉ chia hết cho 1 và chính nó. Nếu danh sách rỗng thì trả về 0.
Ví dụ: Cho danh sách 3, 7, 10, 4, 9 với **head** là phần tử có giá trị 3. Ta gọi phương thức **countPrime()** thì kết quả trả về là 2 (gồm 3 và 7).

Sinh viên tự định nghĩa phương thức in danh sách ra màn hình và định nghĩa lớp **Test** chứa phương thức **main** để kiểm tra lại bài làm.

Câu 2 (3 điểm): Trong thư mục **Cau2** chứa sẵn file **Cau2.java**, trong lớp **Cau2** sinh viên định nghĩa phương thức **public static int recur(int n)** để giải bài toán sau bằng phương pháp **đệ quy** và trả về kết quả.

$$A(n) = \begin{cases} 1, & n=0 \\ 2, & n=1 \\ A(n-1)+2 \cdot A(n-2)+n, & n \geq 2 \end{cases} \quad \text{Với } n \text{ là số nguyên và } n \geq 0$$

Ví dụ: recur(2) trả về 6, recur(3) trả về 13, recur(4) trả về 29.

Sinh viên tự hiện thực trong phương thức **main** để kiểm tra lại bài làm. Lưu ý phương thức **main** gây lỗi cho bài làm thì sinh viên bị 0 điểm cả bài.

Câu 3 (3 điểm): Trong thư mục **Cau3** chứa sẵn file **Cau3.java**, trong lớp **Cau3** sinh viên định nghĩa phương thức **public static int process(String[] ops)** dùng **Stack** để xử lý mảng lệnh theo giải thuật mô tả bên dưới.

Tại câu này, sinh viên được phép sử dụng thư viện **Stack** có sẵn của Java. Câu lệnh **import java.util.Stack;** đã có sẵn trong file **Cau3.java**.

Cho một mảng **String**, mỗi phần tử là một số nguyên không âm hoặc một trong ba lệnh “C”, “D”, “S”. Sinh viên xử lý và trả về kết quả theo giải thuật sau:

1. Tạo một **Stack<Integer>**.
2. Xét lần lượt từng phần tử trong mảng **String**:
 - 2.1. Nếu phần tử là số thì đưa vào **Stack**.
 - 2.2. Nếu phần tử là “C” thì lấy phần tử trên cùng ra khỏi **Stack** (bỏ phần tử này đi).
 - 2.3. Nếu phần tử là “D” thì xem giá trị phần tử trên cùng của **Stack** (không lấy ra), nhân đôi giá trị này rồi đưa kết quả vào **Stack**.
 - 2.4. Nếu phần tử là “S” thì gọi phần tử trên cùng là **o1**, phần tử ngay bên dưới **o1** là **o2**, ta tính **o3 = o1 + o2** rồi đưa **o3** vào **Stack**. Lưu ý **o1** và **o2** vẫn được giữ lại trong **Stack**.
3. Kết quả trả về là tổng các phần tử còn lại trong **Stack**. Nếu **Stack** rỗng thì trả về 0.

Dữ liệu vào luôn hợp lệ: khi gặp lệnh “C” hoặc “D” thì **Stack** có ít nhất 1 phần tử, khi gặp lệnh “S” thì **Stack** có ít nhất 2 phần tử.

Ví dụ cho mảng **String** gồm các phần tử: “5”, “2”, “C”, “D”, “S” (trong cột **Stack**, phần tử bên trái là phần tử trên cùng):

Duyệt mảng	Stack	Tính toán
5	5	
2	2 5	
C	5	Bỏ 2
D	10 5	$5 \times 2 = 10$
S	15 10 5	$10 + 5 = 15$
Kết quả trả về là $15 + 10 + 5 = 30$		

Phương thức **private static boolean isNumber(String str)** dùng để kiểm tra một phần tử **String** có phải là số hay không **đã được cho sẵn** trong file **Cau3.java**, sinh viên sử dụng trực tiếp phương thức này:

```
private static boolean isNumber(String str) {  
    return str.matches("0|([1-9][0-9]*)");  
}
```

Sinh viên tự hiện thực trong phương thức **main** để kiểm tra lại bài làm. Lưu ý phương thức **main** gây lỗi cho bài làm thì sinh viên bị 0 điểm cả bài.